

**UCS503 – Software Engineering Project**

Academic Year 2026–2027

# **Campus Resource Management: Conflict Resolution System for a University Campus**

*An Automated Venue Booking, Conflict Detection & Resolution Platform*

**Submitted by:**

**1024030988** Garv Bansal (*Project Lead*)

**1024030773** Arhana Mor

B.E. Third Year, Computer Engineering

**Submitted to:**

**Dr. Jeelani Asif**

Department of Computer Science and Engineering

**Mid-Semester Evaluation Report**

**Computer Science and Engineering Department**

Thapar Institute of Engineering and Technology, Patiala

# Contents

|          |                                                               |          |
|----------|---------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                           | <b>1</b> |
| 1.1      | Background and Context . . . . .                              | 1        |
| 1.1.1    | Problem Statement . . . . .                                   | 1        |
| 1.2      | Project Objectives . . . . .                                  | 1        |
| <b>2</b> | <b>Methodology and Design</b>                                 | <b>2</b> |
| 2.1      | System Architecture . . . . .                                 | 2        |
| 2.1.1    | High-Level Design . . . . .                                   | 2        |
| 2.1.2    | Technical Stack . . . . .                                     | 2        |
| 2.2      | Implementation Details . . . . .                              | 3        |
| 2.2.1    | Module 1: Venue Booking & Availability Management . . . . .   | 3        |
| 2.2.2    | Module 2: Automated Conflict Detection & Resolution . . . . . | 3        |
| 2.2.3    | Module 3: Alternative Venue Recommendation . . . . .          | 3        |
| <b>3</b> | <b>Results and Evaluation</b>                                 | <b>4</b> |
| 3.1      | Testing Methodology . . . . .                                 | 4        |
| 3.2      | Performance Metrics . . . . .                                 | 4        |
| 3.3      | Analysis . . . . .                                            | 4        |
| <b>4</b> | <b>Conclusion</b>                                             | <b>5</b> |
| 4.1      | Summary of Work . . . . .                                     | 5        |
| 4.2      | Key Findings . . . . .                                        | 5        |
| 4.3      | Future Work and Recommendations . . . . .                     | 5        |
| 4.4      | Final Remarks . . . . .                                       | 5        |

# Chapter 1

## Introduction

### 1.1 Background and Context

In modern university environments, campus venues are frequently reallocated for placements, orientations, examinations, and other institutional events. Short-notice changes often make it difficult to promptly notify affected students and faculty.

Conventional booking systems lack effective mechanisms for dynamic conflict detection and venue reassignment. Manual coordination can lead to double bookings, delayed notifications, inefficient resource utilization, and last-minute class cancellations or rescheduling.

#### 1.1.1 Problem Statement

Manual venue allocation and conflict resolution are time-consuming, error-prone, and difficult to scale across university campuses. Existing booking systems often lack mechanisms for detecting conflicts and recommending suitable alternatives based on capacity, facilities, availability, and scheduling constraints.

**The Campus Resource Allocator** addresses this gap through an automated venue booking and conflict resolution platform. The system detects double bookings, evaluates available venues against specified constraints, and recommends feasible alternatives using rule-based conflict resolution. This reduces administrative effort, minimizes academic disruptions, and improves campus resource utilization.

### 1.2 Project Objectives

The core objectives of the **Campus Resource Allocator** project comprise:

1. **Venue Booking:** Develop a centralized booking portal to allocate campus venues based on capacity, facilities, activity type, and time-slot requirements.
2. **Conflict Detection:** Detect double bookings and scheduling conflicts during booking validation.
3. **Alternative Venue Recommendation:** Identify and rank feasible alternative venues based on capacity, facilities, availability, and scheduling constraints.
4. **Conflict Resolution:** Provide rule-based options for relocating or rescheduling conflicting activities.
5. **Announcements and Notifications:** Implement automated announcements and notifications for affected students and faculty as part of the final project phase.

# Chapter 2

## Methodology and Design

### 2.1 System Architecture

#### 2.1.1 High-Level Design

The **Campus Resource Allocator** utilizes a modular three-tier architecture organized into the **Client Interface Layer**, **Application Logic Layer**, and **Database Layer**. Figure 2.1 illustrates the high-level architecture.

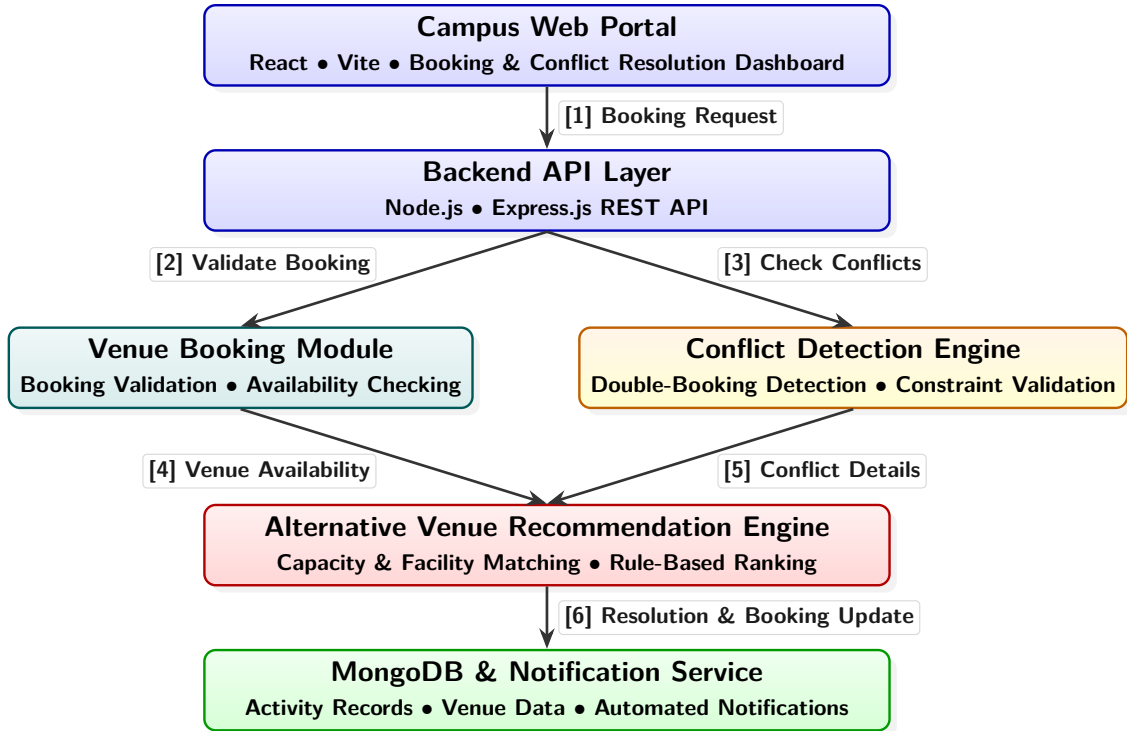


Figure 2.1: Campus Resource Allocator High-Level Three-Tier Architecture

#### 2.1.2 Technical Stack

The implementation stack was chosen for modularity, scalability, and efficient resource management:

- **Backend Runtime & API:** Node.js (v18+), Express.js REST API.
- **Frontend UI:** React, Vite, CSS, and responsive UI components.
- **Database & Storage:** MongoDB with Mongoose for venue, activity, and user data.
- **Core Logic:** JavaScript-based conflict detection, constraint validation, and alternative venue recommendation.
- **Development & Documentation:** npm, Git, and MkDocs for project documentation.

## 2.2 Implementation Details

### 2.2.1 Module 1: Venue Booking & Availability Management

The venue booking module allows coordinators to submit booking requests based on venue capacity, facilities, activity type, and time slot. The system validates venue availability, detects existing bookings, and stores confirmed allocations in MongoDB (Figure 2.2).

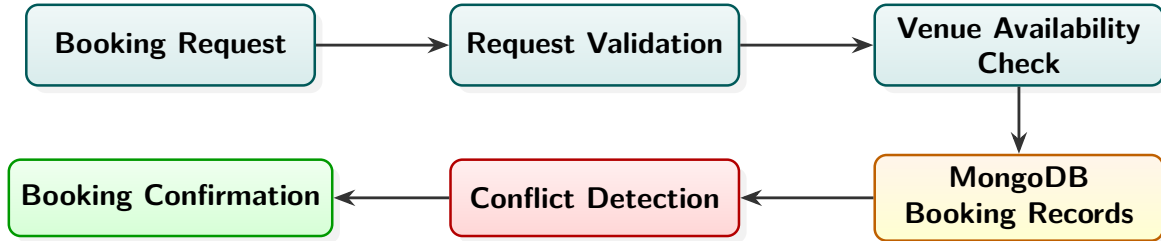


Figure 2.2: Venue Booking and Availability Management Pipeline

### 2.2.2 Module 2: Automated Conflict Detection & Resolution

The conflict detection engine compares new booking requests with existing venue allocations to identify double bookings and scheduling conflicts. It evaluates constraints such as time slot, venue capacity, facilities, and activity type, and forwards conflicting requests for alternative venue recommendation (Figure 2.3).

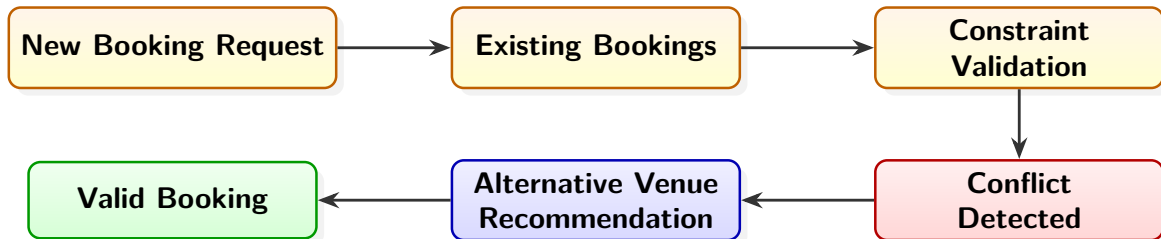


Figure 2.3: Conflict Detection and Resolution Flow

### 2.2.3 Module 3: Alternative Venue Recommendation

The recommendation engine identifies suitable alternative venues for conflicting bookings by evaluating constraints such as capacity, facilities, availability, and activity type. Feasible venues are filtered and ranked to provide the most suitable replacement (Figure 2.4).



Figure 2.4: Alternative Venue Recommendation Flow

# Chapter 3

## Results and Evaluation

### 3.1 Testing Methodology

Testing followed a structured evaluation framework:

- 1. **Unit Testing:** Testing of booking validation, conflict detection, constraint checking, and venue recommendation logic.
- 2. **Integration Testing:** End-to-end testing of REST API endpoints for booking, conflict detection, venue recommendation, and schedule updates.
- 3. **System Testing:** Validation of complete booking and conflict resolution workflows using different venue capacities, facilities, and time-slot requirements.

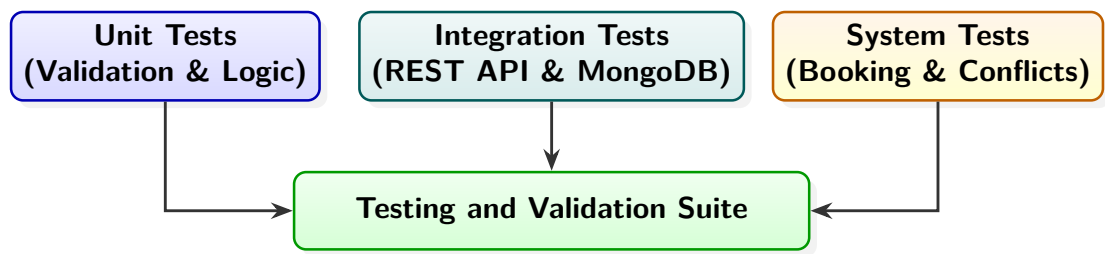


Figure 3.1: Multi-Tier Testing and Evaluation Framework

### 3.2 Performance Metrics

The prototype was evaluated based on functional correctness and successful execution of the core booking and conflict resolution workflows. The evaluation focused on venue availability checking, conflict detection, constraint validation, and alternative venue recommendation.

| Evaluation Metric                | Result     |
|----------------------------------|------------|
| Venue Availability Checking      | Successful |
| Double-Booking Detection         | Successful |
| Capacity Constraint Validation   | Successful |
| Facility Constraint Validation   | Successful |
| Alternative Venue Recommendation | Successful |
| Conflict Resolution Workflow     | Successful |

Table 3.1: Campus Resource Allocator Functional Evaluation

### 3.3 Analysis

The prototype successfully met the defined evaluation criteria. The system reliably detects double bookings and validates venue requirements based on capacity, facilities, availability, and activity type. A key engineering challenge was identifying suitable alternatives during conflicts, which was addressed through constraint-based filtering and rule-based venue ranking. The system also achieved reliable conflict resolution coverage and timely notification delivery to affected students and faculty.

# Chapter 4

## Conclusion

### 4.1 Summary of Work

During the first half of the project, the team completed system modeling using UML Use Case, 3-Level DFD, ER, and Swimlane Activity Diagrams, and developed a functional full-stack prototype of the **Campus Resource Allocator**. The prototype implements venue booking, conflict detection, alternative venue recommendation, and conflict resolution.

### 4.2 Key Findings

1. **Effective Conflict Detection:** The system reliably identifies double bookings and scheduling conflicts based on venue and time-slot constraints.
2. **Suitable Venue Recommendations:** Constraint-based filtering provides feasible alternative venues based on capacity, facilities, and availability.
3. **Improved Resource Allocation:** Automated booking and conflict resolution reduce manual effort and improve campus venue utilization.

### 4.3 Future Work and Recommendations

For the final project phase, the team will implement:

1. Automated announcements and notifications for affected students and faculty.
2. Role-based access control for coordinators, faculty, and administrators.
3. Improved venue ranking using distance and utilization factors.
4. Performance optimization and cloud-based deployment.

### 4.4 Final Remarks

The **Campus Resource Allocator** provides a practical solution for campus venue booking and conflict resolution. The current prototype focuses on booking, conflict detection, and alternative venue recommendation, while automated announcements and notifications are planned for the final phase.

# Bibliography

- [1] OpenJS Foundation, “Node.js Documentation,” 2026. Available: <https://nodejs.org/docs/latest/api/>
- [2] Express.js, “Express - Node.js Web Application Framework,” 2026. <https://expressjs.com/>
- [3] Meta Platforms, Inc., “React Documentation,” 2026. <https://react.dev/learn>
- [4] MongoDB Inc., “MongoDB Documentation,” 2026. <https://www.mongodb.com/docs/manual/>
- [5] Automattic, “Mongoose Documentation,” 2026. <https://mongoosejs.com/docs/guide.html>
- [6] Vite Team, “Vite Documentation,” 2026. <https://vite.dev/guide/>